

Short Course in MATLAB

Lecture 1

Amit Harlev

Learning to code in the age of LLMs

- LLMs are great at coding. You should probably use them in practice.

Learning to code in the age of LLMs

- LLMs are great at coding. You should probably use them in practice.
- Still many reasons to learn to code:
 - ▶ Knowing how to code will help you write better code, even with an LLM.

Learning to code in the age of LLMs

- LLMs are great at coding. You should probably use them in practice.
- Still many reasons to learn to code:
 - ▶ Knowing how to code will help you write better code, even with an LLM.
 - ▶ You still need to be able to read, understand, and explain code.

Learning to code in the age of LLMs

- LLMs are great at coding. You should probably use them in practice.
- Still many reasons to learn to code:
 - ▶ Knowing how to code will help you write better code, even with an LLM.
 - ▶ You still need to be able to read, understand, and explain code.
 - ▶ Teaches you to [think algorithmically](#).

Learning to code in the age of LLMs

- LLMs are great at coding. You should probably use them in practice.
- Still many reasons to learn to code:
 - ▶ Knowing how to code will help you write better code, even with an LLM.
 - ▶ You still need to be able to read, understand, and explain code.
 - ▶ Teaches you to [think algorithmically](#).
 - ▶ Writing code yourself is a great way to understand concepts in other fields such as math or physics.

Learning to code in the age of LLMs

- LLMs are great at coding. You should probably use them in practice.
- Still many reasons to learn to code:
 - ▶ Knowing how to code will help you write better code, even with an LLM.
 - ▶ You still need to be able to read, understand, and explain code.
 - ▶ Teaches you to [think algorithmically](#).
 - ▶ Writing code yourself is a great way to understand concepts in other fields such as math or physics.
 - ▶ It's fun!

LLM use in this class

Example encouraged uses for LLMs

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"
- "What does this specific piece of syntax mean?"

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"
- "What does this specific piece of syntax mean?"

Example inappropriate uses for LLMs

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"
- "What does this specific piece of syntax mean?"

Example inappropriate uses for LLMs

- Any direct modification of your code by an LLM

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"
- "What does this specific piece of syntax mean?"

Example inappropriate uses for LLMs

- Any direct modification of your code by an LLM
- "How do I write a function that.."

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"
- "What does this specific piece of syntax mean?"

Example inappropriate uses for LLMs

- Any direct modification of your code by an LLM
- "How do I write a function that.."
- "Here is my code, check it for any bugs"

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"
- "What does this specific piece of syntax mean?"

Example inappropriate uses for LLMs

- Any direct modification of your code by an LLM
- "How do I write a function that.."
- "Here is my code, check it for any bugs"
- "Explain this function/block of code to me"

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"
- "What does this specific piece of syntax mean?"

Example inappropriate uses for LLMs

- Any direct modification of your code by an LLM
- "How do I write a function that.."
- "Here is my code, check it for any bugs"
- "Explain this function/block of code to me"

In this class we will focus on thinking about code, reading code, and doing cool projects. Things like remembering exact syntax are less important, as it is easy to look up when needed.

LLM use in this class

Example encouraged uses for LLMs

- "What is the MATLAB syntax for..."
- "Explain this concept to me"
- "How do I save to a file in MATLAB?"
- "What does this specific piece of syntax mean?"

Example inappropriate uses for LLMs

- Any direct modification of your code by an LLM
- "How do I write a function that.."
- "Here is my code, check it for any bugs"
- "Explain this function/block of code to me"

In this class we will focus on thinking about code, reading code, and doing cool projects. Things like remembering exact syntax are less important, as it is easy to look up when needed.

I *might* allow full use of LLMs for the final project (I have not yet decided).

Why learn MATLAB

- Good for applied mathematics, computational physics, engineering, etc.

Why learn MATLAB

- Good for applied mathematics, computational physics, engineering, etc.
- Real reason: Many engineering companies and labs have large codebases in MATLAB developed over many years. If you work with one of these, you will need to be familiar with MATLAB.

Why learn MATLAB

- Good for applied mathematics, computational physics, engineering, etc.
- Real reason: Many engineering companies and labs have large codebases in MATLAB developed over many years. If you work with one of these, you will need to be familiar with MATLAB.
- Certain engineering classes might use it.

Why learn MATLAB

- Good for applied mathematics, computational physics, engineering, etc.
- Real reason: Many engineering companies and labs have large codebases in MATLAB developed over many years. If you work with one of these, you will need to be familiar with MATLAB.
- Certain engineering classes might use it.
- However, this course will be valuable to you even if you will never use MATLAB again!

Why learn MATLAB

- Good for applied mathematics, computational physics, engineering, etc.
- Real reason: Many engineering companies and labs have large codebases in MATLAB developed over many years. If you work with one of these, you will need to be familiar with MATLAB.
- Certain engineering classes might use it.
- However, this course will be valuable to you even if you will never use MATLAB again!
- Everything you learn in this course (except syntax) will transfer to other settings (especially with LLMs available to help translate syntax).

Course Logistics

- 3 lectures a week (MWF), for 7 weeks.

Course Logistics

- 3 lectures a week (MWF), for 7 weeks.
- This class is S/U grading only. I don't intend for this class to be stressful.

Course Logistics

- 3 lectures a week (MWF), for 7 weeks.
- This class is S/U grading only. I don't intend for this class to be stressful.
- 1 project/assignment a week

Course Logistics

- 3 lectures a week (MWF), for 7 weeks.
- This class is S/U grading only. I don't intend for this class to be stressful.
- 1 project/assignment a week
 - ▶ Potentially a longer project in the last 2 weeks or going into “finals week” (Oct. 15–22)

Course Logistics

- 3 lectures a week (MWF), for 7 weeks.
- This class is S/U grading only. I don't intend for this class to be stressful.
- 1 project/assignment a week
 - ▶ Potentially a longer project in the last 2 weeks or going into “finals week” (Oct. 15–22)
- Occasional short in-class quizzes, usually on Fridays

Course Logistics

- 3 lectures a week (MWF), for 7 weeks.
- This class is S/U grading only. I don't intend for this class to be stressful.
- 1 project/assignment a week
 - ▶ Potentially a longer project in the last 2 weeks or going into “finals week” (Oct. 15–22)
- Occasional short in-class quizzes, usually on Fridays
 - ▶ Really truly should be no problem if you did the homework and understood what you were doing.

Course Logistics

- 3 lectures a week (MWF), for 7 weeks.
- This class is S/U grading only. I don't intend for this class to be stressful.
- 1 project/assignment a week
 - ▶ Potentially a longer project in the last 2 weeks or going into “finals week” (Oct. 15–22)
- Occasional short in-class quizzes, usually on Fridays
 - ▶ Really truly should be no problem if you did the homework and understood what you were doing.
- A short final project presentation/demo during “finals week”

Course Logistics

- 3 lectures a week (MWF), for 7 weeks.
- This class is S/U grading only. I don't intend for this class to be stressful.
- 1 project/assignment a week
 - ▶ Potentially a longer project in the last 2 weeks or going into “finals week” (Oct. 15–22)
- Occasional short in-class quizzes, usually on Fridays
 - ▶ Really truly should be no problem if you did the homework and understood what you were doing.
- A short final project presentation/demo during “finals week”
- The first homework assignment should be out by Wednesday night and will be due the following Wednesday at 11:59 p.m. on Gradescope/Canvas.

More Logistics

- Best way to reach me is by email (ah843@cornell.edu). Don't hesitate to reach out!

More Logistics

- Best way to reach me is by email (ah843@cornell.edu). Don't hesitate to reach out!
- Office hours once a week, probably on Tuesdays (information will be on Canvas)
 - ▶ OH will start next week

More Logistics

- Best way to reach me is by email (ah843@cornell.edu). Don't hesitate to reach out!
- Office hours once a week, probably on Tuesdays (information will be on Canvas)
 - ▶ OH will start next week
 - ▶ I try to have some candy/snacks

More Logistics

- Best way to reach me is by email (ah843@cornell.edu). Don't hesitate to reach out!
- Office hours once a week, probably on Tuesdays (information will be on Canvas)
 - ▶ OH will start next week
 - ▶ I try to have some candy/snacks
 - ▶ Can also schedule more time with me if you need by sending me an email.

More Logistics

- Best way to reach me is by email (ah843@cornell.edu). Don't hesitate to reach out!
- Office hours once a week, probably on Tuesdays (information will be on Canvas)
 - ▶ OH will start next week
 - ▶ I try to have some candy/snacks
 - ▶ Can also schedule more time with me if you need by sending me an email.
- My offices:
 - ▶ Gates 331: office hours and meetings
 - ▶ Rhodes 657: where I am most of the time

Grading

Assignments and quizzes will be given a score 1, 2, 3, or 4.

Grading

Assignments and quizzes will be given a score 1, 2, 3, or 4.

- **4 (Complete):** All required components are present and correct. At most, there is a very minor issue.

Grading

Assignments and quizzes will be given a score 1, 2, 3, or 4.

- **4 (Complete):** All required components are present and correct. At most, there is a very minor issue.
- **3 (Satisfactory):** Essential requirements are met, but there are minor errors or omissions.

Grading

Assignments and quizzes will be given a score 1, 2, 3, or 4.

- **4 (Complete):** All required components are present and correct. At most, there is a very minor issue.
- **3 (Satisfactory):** Essential requirements are met, but there are minor errors or omissions.
- **2 (Major Issues):** The work shows considerable progress, but has major errors or is incomplete.

Grading

Assignments and quizzes will be given a score 1, 2, 3, or 4.

- **4 (Complete):** All required components are present and correct. At most, there is a very minor issue.
- **3 (Satisfactory):** Essential requirements are met, but there are minor errors or omissions.
- **2 (Major Issues):** The work shows considerable progress, but has major errors or is incomplete.
- **1 (Incomplete):** The submission is missing, largely incomplete, or shows little meaningful progress.

Grading

Assignments and quizzes will be given a score 1, 2, 3, or 4.

- **4 (Complete):** All required components are present and correct. At most, there is a very minor issue.
- **3 (Satisfactory):** Essential requirements are met, but there are minor errors or omissions.
- **2 (Major Issues):** The work shows considerable progress, but has major errors or is incomplete.
- **1 (Incomplete):** The submission is missing, largely incomplete, or shows little meaningful progress.

Passing

You need at least a 3 on every assignment and quiz, and at least a 4 on half of the assignments/projects to pass.

Grading

Assignments and quizzes will be given a score 1, 2, 3, or 4.

- **4 (Complete):** All required components are present and correct. At most, there is a very minor issue.
- **3 (Satisfactory):** Essential requirements are met, but there are minor errors or omissions.
- **2 (Major Issues):** The work shows considerable progress, but has major errors or is incomplete.
- **1 (Incomplete):** The submission is missing, largely incomplete, or shows little meaningful progress.

Passing

You need at least a 3 on every assignment and quiz, and at least a 4 on half of the assignments/projects to pass.

If you get less than a 3 on something, you will need to redo it. I will also probably chat with you to find out what is going on.

Accessing MATLAB

You can access MATLAB in two ways, both of which are free:

- MATLAB Online via browser—set up Mathworks account using your Cornell NetID email address. No download required.
- Download and install on your machine. Cornell [has online instructions on how to do this](#).

Precourse Survey

Everyone that filled it out said they have at least *some* CS experience.

Precourse Survey

Everyone that filled it out said they have at least *some* CS experience.

Based on that, I'm assuming everyone is at least loosely comfortable with the following concepts:

- Variables (in the programming sense)

Precourse Survey

Everyone that filled it out said they have at least *some* CS experience.

Based on that, I'm assuming everyone is at least loosely comfortable with the following concepts:

- Variables (in the programming sense)
- If/else statements

Precourse Survey

Everyone that filled it out said they have at least *some* CS experience.

Based on that, I'm assuming everyone is at least loosely comfortable with the following concepts:

- Variables (in the programming sense)
- If/else statements
- For and while loops

Precourse Survey

Everyone that filled it out said they have at least *some* CS experience.

Based on that, I'm assuming everyone is at least loosely comfortable with the following concepts:

- Variables (in the programming sense)
- If/else statements
- For and while loops
- You kinda know what a function is (in the programming sense)

Precourse Survey

Everyone that filled it out said they have at least *some* CS experience.

Based on that, I'm assuming everyone is at least loosely comfortable with the following concepts:

- Variables (in the programming sense)
- If/else statements
- For and while loops
- You kinda know what a function is (in the programming sense)
- You kinda know what an array/list/vector is (in the programming sense)

Precourse Survey

Everyone that filled it out said they have at least *some* CS experience.

Based on that, I'm assuming everyone is at least loosely comfortable with the following concepts:

- Variables (in the programming sense)
- If/else statements
- For and while loops
- You kinda know what a function is (in the programming sense)
- You kinda know what an array/list/vector is (in the programming sense)

If this does not describe you, please come chat with me after class! I can help you get caught up.

We will also go through all of these again in MATLAB, but relatively quickly.

Variable declaration

Create variables simply by assigning a value to a name with =.

```
1 x = 5;  
2 name = "Amit";
```

Variable declaration

Create variables simply by assigning a value to a name with `=`.

```
1 x = 5;  
2 name = "Amit";
```

Numbers have type `double` by default. Every numeric value in MATLAB is an array; a single number is a 1×1 array.

Variable declaration

Create variables simply by assigning a value to a name with `=`.

```
1 x = 5;  
2 name = "Amit";
```

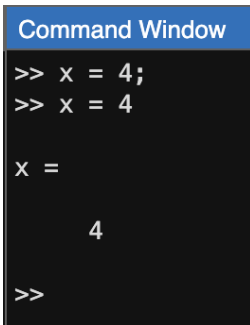
Numbers have type `double` by default. Every numeric value in MATLAB is an array; a single number is a 1×1 array.

Text enclosed in double quotes, such as `"Amit"`, has type `string`. Use double quotes for strings; single quotes create a different type called a character vector.

Semicolons in MATLAB

```
1 x = 5;  
2 name = "Amit";
```

Ending a statement with a semicolon suppresses its output. Without a semicolon, MATLAB displays the result in the Command Window. When writing a script, you will usually use semicolons unless you intentionally want to inspect a result.

A screenshot of the MATLAB Command Window. The window has a blue title bar that says "Command Window". The background is black with white text. The text shows two commands: ">> x = 4;" and ">> x = 4". The first command is followed by the output "x =" and then the value "4" on the next line. The second command is followed by the prompt ">>" on the next line, indicating that the output was suppressed.

```
Command Window  
>> x = 4;  
>> x = 4  
  
x =  
  
    4  
  
>>
```


Assignment stores a value

```
1 x = 5;  
2 y = x + 1; % y equals 6  
3 x = 3; % y still equals 6
```

When MATLAB runs `y = x + 1`, it evaluates the right-hand side immediately and stores the resulting value in `y`.

- Changing `x` later does not cause MATLAB to recalculate `y`.
- This applies to ordinary MATLAB values of every type, including numbers, arrays, and strings.

Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;  
2 y = x - 1; % y equals 4
```

Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;  
2 y = x - 1; % y equals 4  
3 z = x * y; % z equals 20
```

Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;  
2 y = x - 1; % y equals 4  
3 z = x * y; % z equals 20  
4 a = 3 / 4; % a equals 0.75
```

Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;  
2 y = x - 1; % y equals 4  
3 z = x * y; % z equals 20  
4 a = 3 / 4; % a equals 0.75  
5 b = abs(-4); % b equals 4
```

Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;  
2 y = x - 1; % y equals 4  
3 z = x * y; % z equals 20  
4 a = 3 / 4; % a equals 0.75  
5 b = abs(-4); % b equals 4  
6 c = ceil(a); % c equals 1
```

Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;  
2 y = x - 1; % y equals 4  
3 z = x * y; % z equals 20  
4 a = 3 / 4; % a equals 0.75  
5 b = abs(-4); % b equals 4  
6 c = ceil(a); % c equals 1  
7 d = floor(a); % d equals 0
```

Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;  
2 y = x - 1; % y equals 4  
3 z = x * y; % z equals 20  
4 a = 3 / 4; % a equals 0.75  
5 b = abs(-4); % b equals 4  
6 c = ceil(a); % c equals 1  
7 d = floor(a); % d equals 0  
8 e = round(a); % e equals 1
```


Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;
2 y = x - 1; % y equals 4
3 z = x * y; % z equals 20
4 a = 3 / 4; % a equals 0.75
5 b = abs(-4); % b equals 4
6 c = ceil(a); % c equals 1
7 d = floor(a); % d equals 0
8 e = round(a); % e equals 1
9 f = sqrt(b); % f equals 2
```

Math operations

All basic math operations are available in MATLAB:

```
1 x = 5;
2 y = x - 1; % y equals 4
3 z = x * y; % z equals 20
4 a = 3 / 4; % a equals 0.75
5 b = abs(-4); % b equals 4
6 c = ceil(a); % c equals 1
7 d = floor(a); % d equals 0
8 e = round(a); % e equals 1
9 f = sqrt(b); % f equals 2
10 g = 2 ^ 3; % g equals 8
```

Modular arithmetic

Modular arithmetic in MATLAB is done with `mod(a,m)`.

```
1 r = mod(a, m);
```

`r` is the remainder after dividing `a` by `m`.

```
1 x = mod(10, 3);
```

Modular arithmetic

Modular arithmetic in MATLAB is done with `mod(a,m)`.

```
1 r = mod(a, m);
```

`r` is the remainder after dividing `a` by `m`.

```
1 x = mod(10, 3); % x equals 1
```

Modular arithmetic

Modular arithmetic in MATLAB is done with `mod(a,m)`.

```
1 r = mod(a, m);
```

`r` is the remainder after dividing `a` by `m`.

```
1 x = mod(10, 3); % x equals 1  
2 y = mod(16, 2); % What is y?
```

Modular arithmetic

Modular arithmetic in MATLAB is done with `mod(a,m)`.

```
1 r = mod(a, m);
```

`r` is the remainder after dividing `a` by `m`.

```
1 x = mod(10, 3); % x equals 1
2 y = mod(16, 2); % What is y?
3 z = mod(17, 12); % What is z?
```

Arrays in MATLAB

A 1-d array is an ordered collection of values stored under one name.

```
1 scores = [8, 10, 7, 9];
```

Arrays in MATLAB

A 1-d array is an ordered collection of values stored under one name.

```
1 scores = [8, 10, 7, 9];
```

We can access the k th entry in the array using `scores(k)`

```
1 x = scores(1); % what is x?  
2 y = scores(3); % what is y?
```


Arrays in MATLAB

A 1-d array is an ordered collection of values stored under one name.

```
1 scores = [8, 10, 7, 9];
```

We can access the k th entry in the array using `scores(k)`

```
1 x = scores(1); % what is x?  
2 y = scores(3); % what is y?
```

We can also change values after accessing them.

```
1 scores(3) = -3 % scores is now [8, 11, -3, 9]  
2           % what is y now?
```

Arrays in MATLAB

A 1-d array is an ordered collection of values stored under one name.

```
1 scores = [8, 10, 7, 9];
```

We can access the k th entry in the array using `scores(k)`

```
1 x = scores(1); % what is x?  
2 y = scores(3); % what is y?
```

We can also change values after accessing them.

```
1 scores(3) = -3 % scores is now [8, 11, -3, 9]  
2           % what is y now?
```

- Use square brackets, `[ ]`, to create an array.
- Use parentheses, `( )`, to access an element and change it.

Creating arrays in MATLAB

There are many different ways to create an array. All of the following create the same array:

```
1 a = [1, 2, 3, 4, 5];
```

Creating arrays in MATLAB

There are many different ways to create an array. All of the following create the same array:

```
1 a = [1, 2, 3, 4, 5];  
2 b = [1 2 3 4 5];
```

Creating arrays in MATLAB

There are many different ways to create an array. All of the following create the same array:

```
1 a = [1, 2, 3, 4, 5];  
2 b = [1 2 3 4 5];  
3 c = 1:5;
```

Creating arrays in MATLAB

There are many different ways to create an array. All of the following create the same array:

```
1 a = [1, 2, 3, 4, 5];  
2 b = [1 2 3 4 5];  
3 c = 1:5;  
4 d = linspace(1, 5, 5);
```

Creating arrays in MATLAB

There are many different ways to create an array. All of the following create the same array:

```
1 a = [1, 2, 3, 4, 5];  
2 b = [1 2 3 4 5];  
3 c = 1:5;  
4 d = linspace(1, 5, 5);
```

More examples of the colon operator:

```
1 4:11; % [4 5 6 7 8 9 10 11]
```

Creating arrays in MATLAB

There are many different ways to create an array. All of the following create the same array:

```
1 a = [1, 2, 3, 4, 5];  
2 b = [1 2 3 4 5];  
3 c = 1:5;  
4 d = linspace(1, 5, 5);
```

More examples of the colon operator:

```
1 4:11; % [4 5 6 7 8 9 10 11]  
2 4:3:11; % [4 7 10]
```


Creating arrays in MATLAB

There are many different ways to create an array. All of the following create the same array:

```
1 a = [1, 2, 3, 4, 5];  
2 b = [1 2 3 4 5];  
3 c = 1:5;  
4 d = linspace(1, 5, 5);
```

More examples of the colon operator:

```
1 4:11; % [4 5 6 7 8 9 10 11]  
2 4:3:11; % [4 7 10]  
3 7:-2:0; % [7 5 3 1]
```

Creating even more arrays in MATLAB

More examples of `linspace`:

```
1 linspace(10, 30, 5);
```

Creating even more arrays in MATLAB

More examples of `linspace`:

```
1 linspace(10, 30, 5); % [10 15 20 25 30]
```

Creating even more arrays in MATLAB

More examples of `linspace`:

```
1 linspace(10, 30, 5); % [10 15 20 25 30]
2 linspace(1, 2, 5); % [1 1.25 1.5 1.75 2]
```

Creating even more arrays in MATLAB

More examples of `linspace`:

```
1 linspace(10, 30, 5); % [10 15 20 25 30]
2 linspace(1, 2, 5); % [1 1.25 1.5 1.75 2]
3 linspace(-6, -17, 5); % [-6 -8.75 -11.5 -14.25 -17]
```

Creating even more arrays in MATLAB

More examples of `linspace`:

```
1 linspace(10, 30, 5); % [10 15 20 25 30]  
2 linspace(1, 2, 5); % [1 1.25 1.5 1.75 2]  
3 linspace(-6, -17, 5); % [-6 -8.75 -11.5 -14.25 -17]
```

Here are even more ways to create arrays:

```
1 z = zeros(1, 3); % z is [0 0 0]
```

Creating even more arrays in MATLAB

More examples of `linspace`:

```
1 linspace(10, 30, 5); % [10 15 20 25 30]  
2 linspace(1, 2, 5); % [1 1.25 1.5 1.75 2]  
3 linspace(-6, -17, 5); % [-6 -8.75 -11.5 -14.25 -17]
```

Here are even more ways to create arrays:

```
1 z = zeros(1, 3); % z is [0 0 0]  
2 n = ones(1, 4); % n is [1 1 1 1]
```

Creating even more arrays in MATLAB

More examples of `linspace`:

```
1 linspace(10, 30, 5); % [10 15 20 25 30]  
2 linspace(1, 2, 5); % [1 1.25 1.5 1.75 2]  
3 linspace(-6, -17, 5); % [-6 -8.75 -11.5 -14.25 -17]
```

Here are even more ways to create arrays:

```
1 z = zeros(1, 3); % z is [0 0 0]  
2 n = ones(1, 4); % n is [1 1 1 1]  
3 r = rand(1, 2); % two random numbers between 0 and 1
```


Colon operator for indexing

The colon operator can also be used to index into an array:

```
1 A = [2 -3 7 7 1 0 1];  
2 A(1:3);
```

Colon operator for indexing

The colon operator can also be used to index into an array:

```
1 A = [2 -3 7 7 1 0 1];  
2 A(1:3); % elements 1 through 3: [2 -3 7]
```

Colon operator for indexing

The colon operator can also be used to index into an array:

```
1 A = [2 -3 7 7 1 0 1];  
2 A(1:3); % elements 1 through 3: [2 -3 7]  
3 A(3:2:7);
```

Colon operator for indexing

The colon operator can also be used to index into an array:

```
1 A = [2 -3 7 7 1 0 1];  
2 A(1:3); % elements 1 through 3: [2 -3 7]  
3 A(3:2:7); % elements 3, 5, and 7: [7 1 1]
```

Colon operator for indexing

The colon operator can also be used to index into an array:

```
1 A = [2 -3 7 7 1 0 1];  
2 A(1:3); % elements 1 through 3: [2 -3 7]  
3 A(3:2:7); % elements 3, 5, and 7: [7 1 1]  
4 A(7:-1:1);
```

Colon operator for indexing

The colon operator can also be used to index into an array:

```
1 A = [2 -3 7 7 1 0 1];  
2 A(1:3); % elements 1 through 3: [2 -3 7]  
3 A(3:2:7); % elements 3, 5, and 7: [7 1 1]  
4 A(7:-1:1); % reverse order: [1 0 1 7 7 -3 2]
```

For loops in MATLAB

A **for** loop runs the same block of code once for each value in a sequence.

```
1 total = 0;
2
3 for k = 1:4
4     total = total + k;
5 end
6 % what does total equal here?
```

- The loop runs four times, with **k** taking each value in **[1, 2, 3, 4]** in order.
- **end** marks the end of the repeated code.

For loops in MATLAB (vs e.g. Java)

```
1 for i = 2:5
2     disp(i); % prints to command window
3     i = 4;
4     disp(i);
5 end
```

What will this print out?

For loops in MATLAB (vs e.g. Java)

```
1 for i = 2:5
2     disp(i); % prints to command window
3     i = 4;
4     disp(i);
5 end
```

What will this print out?

2 4

For loops in MATLAB (vs e.g. Java)

```
1 for i = 2:5
2     disp(i); % prints to command window
3     i = 4;
4     disp(i);
5 end
```

What will this print out?

2 4 3 4 4 4 5 4

Demo: $1 + 1/4 + 1/9 + 1/16 + \dots$

How might we compute this sum in MATLAB? (up to, say, the 1,000,000th term)

Demo: $1 + 1/4 + 1/9 + 1/16 + \dots$

How might we compute this sum in MATLAB? (up to, say, the 1,000,000th term)

```
1 total = 0;
2 for i = 1:1000000
3     total = total + 1/i^2;
4 end
5 disp(sqrt(6 * total)); % any guesses for what this is?
```

Demo: $1 + 1/4 + 1/9 + 1/16 + \dots$

How might we compute this sum in MATLAB? (up to, say, the 1,000,000th term)

```
1 total = 0;
2 for i = 1:1000000
3     total = total + 1/i^2;
4 end
5 disp(sqrt(6 * total)); % any guesses for what this is?
```

This will print the value 3.141591698660509.

Demo: $1 + 1/4 + 1/9 + 1/16 + \dots$

How might we compute this sum in MATLAB? (up to, say, the 1,000,000th term)

```
1 total = 0;
2 for i = 1:1000000
3     total = total + 1/i^2;
4 end
5 disp(sqrt(6 * total)); % any guesses for what this is?
```

This will print the value 3.141591698660509.

Fun fact

$$1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16} + \dots = \frac{\pi^2}{6}.$$

Demo cont.

Can we see how quickly this approaches π ?

Demo cont.

Can we see how quickly this approaches π ?

We need to keep track of total after each iteration of the loop:

```
1 totals = [1];  
2 for i = 2:10000000  
3     totals(i) = totals(i-1) + 1/i^2;  
4 end
```

Note the new usage of the `( )` syntax: we are growing the array by accessing an index larger than the current size of the array.

Demo cont.

Can we see how quickly this approaches π ?

We need to keep track of total after each iteration of the loop:

```
1 totals = [1];  
2 for i = 2:10000000  
3     totals(i) = totals(i-1) + 1/i^2;  
4 end
```

Note the new usage of the `( )` syntax: we are growing the array by accessing an index larger than the current size of the array.

In MATLAB, one would usually *preallocate* the array because growing an array one element at a time is slow for the computer to do.

Demo cont.

Can we see how quickly this approaches π ?

We need to keep track of total after each iteration of the loop:

```
1 n = 1000000;  
2 totals = zeros(1, n);  
3 totals(1) = 1;  
4 for i = 2:n  
5     totals(i) = totals(i-1) + 1/i^2;  
6 end
```

Note the new usage of the `( )` syntax: we are growing the array by accessing an index larger than the current size of the array.

In MATLAB, one would usually *preallocate* the array because growing an array one element at a time is slow for the computer.

Demo cont.

Can we see how quickly this approaches π ?

```
1 n = 1000000;  
2 totals = zeros(1, n);  
3 totals(1) = 1;  
4 for i = 2:n  
5     totals(i) = totals(i-1) + 1/i^2;  
6 end  
7  
8 pi_estimates = zeros(1, n);  
9 for i = 1:n  
10    pi_estimates(i) = sqrt(totals(i) * 6);  
11 end
```

Demo cont.

Can we see how quickly this approaches π ?

```
1 n = 1000000;  
2 totals = zeros(1, n);  
3 totals(1) = 1;  
4 for i = 2:n  
5     totals(i) = totals(i-1) + 1/i^2;  
6 end  
7  
8 pi_estimates = zeros(1, n);  
9 for i = 1:n  
10     pi_estimates(i) = sqrt(totals(i) * 6);  
11 end  
12  
13 figure  
14 plot(pi_estimates);  
15 yline(pi, '--');
```

More on plotting in MATLAB in the future!